JavaScript Array Methods

Quick reference for React + TypeScript | Methods marked “mutate” change the original array.

TRANSFORM & ITERATE

METHOD	WHAT IT DOES	EXAMPLE / RESULT
`map(fn)`	Transform every item	`[1,2].map(x => x*2) // [2,4]`
`filter(fn)`	Keep matching items	`[1,2,3].filter(x => x>1) // [2,3]`
`reduce(fn, init)`	Accumulate one result	`[1,2,3].reduce((a,b)=>a+b,0) // 6`
`forEach(fn)`	Run callback; returns undefined	`[1,2].forEach(x => log(x))`
`flat(depth?)`	Flatten nested arrays	`[[1],[2,3]].flat() // [1,2,3]`
`flatMap(fn)`	Map then flatten one level	`[1,2].flatMap(x => [x,x]) // [1,1,2,2]`

FIND & TEST

METHOD	WHAT IT DOES	EXAMPLE / RESULT
`find(fn)`	First matching item or undefined	`[2,4,6].find(x => x>3) // 4`
`findIndex(fn)`	First matching index or -1	`[2,4,6].findIndex(x => x>3) // 1`
`some(fn)`	True if at least one matches	`[1,2,3].some(x => x>2) // true`
`every(fn)`	True if all match	`[1,2,3].every(x => x>0) // true`
`includes(v)`	Whether value exists	`[1,2].includes(2) // true`
`indexOf(v)`	First index of value or -1	`[4,7].indexOf(7) // 1`

CHANGE & COPY

METHOD	WHAT IT DOES	EXAMPLE / RESULT
`push(v) / pop()`	Add/remove at end; mutate	`a.push(3); a.pop()`
`unshift(v) / shift()`	Add/remove at start; mutate	`a.unshift(0); a.shift()`
`splice(i,n,...)`	Insert/remove in place; mutate	`a.splice(1,1) // remove index 1`
`slice(start,end)`	Copy portion; end exclusive	`[1,2,3].slice(0,2) // [1,2]`
`concat(other)`	Join arrays; new array	`[1].concat([2]) // [1,2]`
`sort() / reverse()`	Sort/reverse in place; mutate	`[3,1,2].sort((a,b)=>a-b)`
`toSorted() / toReversed()`	Non-mutating alternatives	`[3,1].toSorted((a,b)=>a-b)`

EXTRAS

METHOD	WHAT IT DOES	EXAMPLE / RESULT
`join(separator)`	Convert array to string	`["A","B"].join("-") // "A-B"`
`at(index)`	Get item; negative from end	`[1,2,3].at(-1) // 3`
`fill(value)`	Replace slots; mutate	`new Array(3).fill(0) // [0,0,0]`
`Array.from(x)`	Make array from iterable	`Array.from("AB") // ["A","B"]`
`Array.isArray(x)`	Check array type	`Array.isArray([]) // true`
`Array.of(...)`	Array from arguments	`Array.of(3) // [3]`

React tip: Prefer non-mutating methods when updating state: `map`, `filter`, `slice`, `toSorted`. For changes to nested arrays, copy each level you modify.

Memory aid: `some` = any, `every` = all (there is no Array.all). `map` = same number of elements, transformed; `filter` = subset; `reduce` = one accumulated result.